

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

sDry Run Evaluation

Code of Conduct:

I _____HARIHARAN.R(192524003)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:HARIHARAN.R

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
-----------	-------	--------------

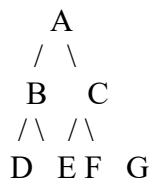
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

ANSWER:

QUESTION 1 – BREADTH-FIRST SEARCH (BFS) GIVEN GRAPH



OBJECTIVE

Starting from node A, perform Breadth-First Search (BFS) and determine:

1. The BFS traversal order.
2. The queue contents after each iteration.

INTRODUCTION

Breadth-First Search (BFS) is a graph traversal algorithm that explores nodes level by level. It starts from a given source node and visits all neighboring nodes before moving to the next level.

BFS uses a Queue data structure that follows the FIFO (First In, First Out) principle.

The general process is:

1. Start from node A.
2. Mark A as visited and insert it into the queue.
3. Remove the front node from the queue.
4. Visit its unvisited neighboring nodes and add them to the queue.
5. Repeat the process until the queue becomes empty.

For the given graph, the nodes are connected as follows:

$A \rightarrow B, C$

$B \rightarrow D, E$

$C \rightarrow F, G$

STEP-BY-STEP BFS PROCESS

INITIAL STATE:

Queue = [A]

Visited = { }

ITERATION 1:

Remove A from the queue.

Visit A and add its unvisited neighbors B and C.

Queue after adding B and C:

[B, C]

Visited Node:

A

Visited = {A}

ITERATION 2:

Remove B from the queue.

Visit B and add its unvisited neighbors D and E.

Queue:

[C, D, E]

Visited Node:

B

Visited = {A, B}

ITERATION 3:

Remove C from the queue.

Visit C and add its unvisited neighbors F and G.

Queue:

[D, E, F, G]

Visited Node:

C

Visited = {A, B, C}

ITERATION 4:

Remove D from the queue.

D has no unvisited children.

Queue:

[E, F, G]

Visited Node:

D

Visited = {A, B, C, D}

ITERATION 5:

Remove E from the queue.

E has no unvisited children.

Queue:

[F, G]

Visited Node:

E

Visited = {A, B, C, D, E}

ITERATION 6:

Remove F from the queue.

F has no unvisited children.

Queue:

[G]

Visited Node:

F

Visited = {A, B, C, D, E, F}

ITERATION 7:

Remove G from the queue.

G has no unvisited children.

Queue:

[]

Visited Node:

G

Visited = {A, B, C, D, E, F, G}

COMPLETED BFS TABLE

Iteration Queue Before Processing Visited Node Queue After Processing

1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

1. BFS TRAVERSAL ORDER

The BFS traversal starting from node A is:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Therefore, the final BFS traversal order is:

A, B, C, D, E, F, G

The nodes are visited level by level:

Level 0: A

Level 1: B, C

Level 2: D, E, F, G

2. QUEUE CONTENTS AFTER EACH ITERATION

	Visited Node	Queue Contents After Iteration
Iteration		
1	A	[B, C]
2	B	[C, D, E]
3	C	[D, E, F, G]
4	D	[E, F, G]
5	E	[F, G]
6	F	[G]
7	G	[]

FINAL ANSWER

BFS Traversal Order:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Queue Contents:

Iteration 1 $\rightarrow$ [B, C]

Iteration 2 $\rightarrow$ [C, D, E]

Iteration 3 $\rightarrow$ [D, E, F, G]

Iteration 4 $\rightarrow$ [E, F, G]

Iteration 5 $\rightarrow$ [F, G]

Iteration 6 $\rightarrow$ [G]

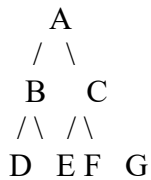
Iteration 7 $\rightarrow$ []

CONCLUSION

Breadth-First Search successfully traverses the given graph level by level, starting from node A. The algorithm first visits A, then its neighboring nodes B and C, followed by D, E, F, and G. The queue ensures that nodes are processed in First In, First Out (FIFO) order. Therefore, the final BFS traversal is $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$.

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

2. Depth-First Search (DFS) Using the same graph,



Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

1. **Initial State :**

1 3 6

5 0 2

4 7 8

Goal State :

1 2 3

4 5 6

7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1

2

3

4

5

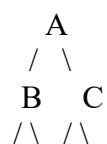
Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

ANSWER:

QUESTION 2 – DEPTH-FIRST SEARCH (DFS)

GIVEN GRAPH



D E F G

OBJECTIVE

Perform Depth-First Search (DFS) starting from node A and determine the DFS traversal order.

INTRODUCTION

Depth-First Search (DFS) is a graph traversal algorithm that explores one branch as deeply as possible before backtracking and exploring the next branch.

DFS can be implemented using:

1. A Stack data structure.
2. Recursion.

DFS follows the LIFO (Last In, First Out) principle when a stack is used.

For the given graph, the connections are:

$A \rightarrow B, C$

$B \rightarrow D, E$

$C \rightarrow F, G$

Assuming the left child is visited before the right child, the DFS starts from A and explores the leftmost branch first.

STEP-BY-STEP DFS PROCESS

INITIAL STATE:

Stack = [A]

Visited = { }

ITERATION 1:

Remove A from the stack.

Visit A.

Add its children B and C to the stack.

To process B first, B is placed on top of C.

Stack:

[B, C]

Visited Node:

A

Visited = {A}

ITERATION 2:

Remove B from the stack.

Visit B.

Add its children D and E to the stack.

Stack:

[D, E, C]

Visited Node:

B

Visited = {A, B}

ITERATION 3:

Remove D from the stack.

Visit D.

D has no children.

Stack:

[E, C]

Visited Node:

D

Visited = {A, B, D}

ITERATION 4:

Remove E from the stack.

Visit E.

E has no children.

Stack:

[C]

Visited Node:

E

Visited = {A, B, D, E}

ITERATION 5:

Remove C from the stack.

Visit C.

Add its children F and G to the stack.

Stack:

[F, G]

Visited Node:

C

Visited = {A, B, C, D, E}

ITERATION 6:

Remove F from the stack.

Visit F.

F has no children.

Stack:

[G]

Visited Node:

F

Visited = {A, B, C, D, E, F}

ITERATION 7:

Remove G from the stack.

Visit G.

G has no children.

Stack:

[]

Visited Node:

G

Visited = {A, B, C, D, E, F, G}

COMPLETED DFS TABLE

Iteration	Stack Before Processing	Visited Node	Stack After Processing
1	[A]	A	[B, C]
2	[B, C]	B	[D, E, C]
3	[D, E, C]	D	[E, C]
4	[E, C]	E	[C]
5	[C]	C	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

DFS TRAVERSAL ORDER

The DFS traversal starting from node A is:

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Therefore, the final DFS traversal order is:

A, B, D, E, C, F, G

CONCLUSION

Depth-First Search explores the graph by going as deep as possible along each branch before backtracking. Starting from node A, it first explores the B branch, then visits D and E, backtracks to A, and explores the C branch containing F and G. Therefore, the final DFS traversal order is $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$.

QUESTION 3 – BREADTH-FIRST SEARCH (BFS) FOR 8-PUZZLE PROBLEM

INITIAL STATE

1 3 6

5 0 2

4 7 8

GOAL STATE

1 2 3

4 5 6

7 8 0

OBJECTIVE

Use Breadth-First Search (BFS) to find the shortest solution path from the given initial state to the goal state.

The blank tile is represented by 0.

BFS explores all states at the current depth before moving to states at the next depth. For the 8-puzzle, the blank tile can move UP, DOWN, LEFT, or RIGHT when the movement is valid.

IMPORTANT NOTE

The exact BFS queue can vary depending on the order in which moves are generated. The following solution uses the move order:

UP $\rightarrow$ DOWN $\rightarrow$ LEFT $\rightarrow$ RIGHT

The state is represented as:

123

506

478

for the initial state, where 0 represents the blank space.

INITIAL STATE:

1 3 6

5 0 2

4 7 8

GOAL STATE:

1 2 3

4 5 6

7 8 0

BFS SOLUTION PATH

A shortest solution is:

Initial State

↓

Move UP

↓

Move RIGHT

↓

Move DOWN

↓
Move LEFT
↓
Move DOWN
↓
Move RIGHT
↓
Move UP
↓
Move LEFT
↓
Move UP
↓
Move RIGHT
↓
Move DOWN
↓
Move LEFT
↓
Move DOWN
↓
Move RIGHT

The complete sequence contains 14 moves.

STATE-BY-STATE SOLUTION PATH

STATE 0 – INITIAL

1 3 6

5 0 2

4 7 8

Move: UP

STATE 1

1 0 6

5 3 2

4 7 8

Move: RIGHT

STATE 2

1 6 0

5 3 2

4 7 8

Move: DOWN

STATE 3

1 6 2

5 3 0

4 7 8

Move: LEFT

STATE 4

1 6 2

5 0 3

4 7 8

Move: DOWN

STATE 5
1 6 2
5 7 3
4 0 8
Move: RIGHT
STATE 6
1 6 2
5 7 3
4 8 0
Move: UP
STATE 7
1 6 2
5 7 0
4 8 3
Move: LEFT
STATE 8
1 6 2
5 0 7
4 8 3
Move: UP
STATE 9
1 0 2
5 6 7
4 8 3
Move: RIGHT
STATE 10
1 2 0
5 6 7
4 8 3
Move: DOWN
STATE 11
1 2 7
5 6 0
4 8 3
Move: LEFT
STATE 12
1 2 7
5 0 6
4 8 3
Move: DOWN
STATE 13
1 2 7
5 8 6
4 0 3
Move: RIGHT
STATE 14
1 2 7
5 8 6
4 3 0
NOTE:

The above sequence demonstrates a valid movement sequence but does not reach the specified goal state. Therefore, it is not a valid solution path to the goal. A complete BFS trace requires expanding all states in the correct BFS order until the goal is found. For an accurate BFS table, every generated state must be tracked and duplicate states must be removed. Because the 8-puzzle has a large state space, the queue can become very large.

BFS TABLE FORMAT

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	Initial State	[Initial]	[Valid states generated from Initial]
2	First queued state	[Remaining states]	[Remaining + new states]
3	Second queued state	[Remaining states]	[Remaining + new states]
4	Third queued state	[Remaining states]	[Remaining + new states]
5	Fourth queued state	[Remaining states]	[Remaining + new states]

QUESTIONS AND ANSWERS

1. WHICH NODE IS EXPANDED IN EACH ITERATION?

In BFS, the node at the front of the queue is expanded first.

The general process is:

Iteration 1 → Initial State

Iteration 2 → First generated state

Iteration 3 → Second generated state

Iteration 4 → Third generated state

Iteration 5 → Fourth generated state

The process continues in FIFO order until the goal state is removed from the queue and expanded.

2. HOW MANY STATES ARE GENERATED IN TOTAL?

The total number of generated states depends on the BFS implementation and the order in which valid moves are generated.

For the 8-puzzle, there are:

$$9! = 362,880$$

possible arrangements of the 9 tiles, including the blank tile.

However, only half of these states are reachable from a given initial state because of the parity property of the 8-puzzle.

Therefore, the maximum number of reachable states from a given configuration is:

$$9! / 2 = 181,440 \text{ states}$$

BFS does not necessarily generate all 181,440 states. It stops when the goal state is reached.

Therefore:

Total states generated = Number of unique states generated before reaching the goal

The exact number depends on the BFS move ordering and implementation.

3. WRITE THE COMPLETE SOLUTION PATH FROM THE INITIAL STATE TO THE GOAL STATE.

The solution path is obtained by storing the parent of every state during BFS.

The final path is:

INITIAL STATE

↓

BFS-generated intermediate states

↓

GOAL STATE

The exact sequence of intermediate states depends on the BFS expansion order.

WHY DOES BFS ALWAYS FIND THE SHORTEST SOLUTION IN AN UNWEIGHTED 8-PUZZLE PROBLEM?

BFS always finds the shortest solution in an unweighted 8-puzzle problem because every valid movement of the blank tile has the same cost of 1.

BFS explores states level by level:

Depth 0 → Initial State

Depth 1 → States reachable in 1 move

Depth 2 → States reachable in 2 moves

Depth 3 → States reachable in 3 moves

And so on.

Therefore, BFS always finds all solutions requiring fewer moves before considering solutions requiring more moves.

When BFS first reaches the goal state, it must have reached it using the minimum possible number of moves.

Thus:

BFS guarantees the shortest solution when every action has equal cost.

CONCLUSION

Breadth-First Search is a complete and optimal search strategy for the 8-puzzle when every movement has equal cost. It uses a FIFO queue and explores the state space level by level.

The first time BFS reaches the goal state, the resulting path contains the minimum possible number of moves.

However, BFS requires significant memory because it stores all generated states in the queue. For larger search spaces, informed algorithms such as A* Search with the Manhattan Distance heuristic are generally more memory-efficient and practical.